XGBoost

학번	212STG15	이름	서혜정
학번	212STG33	이름	황예진

Contents

1. Introduction

2. Tree Boosting in a Nutshell
   2.1 Regularized Learning Objective
   2.2 Gradient Tree Boosting

3. Split Finding Algorithms
   3.1 Basic Exact Greedy Algorithm
   3.2 Approximate Algorithm
   3.3 Weighted Quantile Sketch
   3.4 Sparsity-aware Split Finding

4. System Design
   4.1 Column Block for Parallel Learning
   4.2 Cache-aware Access
   4.3 Blocks for Out of core Computation

5. Example using python and R

1. Introduction

XGBoost 는 eXtreme Gradient Boosting 의 약어로서, 기존 gradient boosting 방법에 Extreme 한 요소들을 추가한 모델이다. 이를 통해 더 빠르고 대용량의 데이터 처리가 가능하도록 한다. 대용량 데이터를 처리할 수 있는 특징을 scalability(확장성)이라 한다. XGBoost 는 크게 8 가지의 특징이 있다.

- Regularized Learning Objective Function
- Learning rate (Shrinkage) and Column subsampling
- Approximate Greedy Algorithm
- Weighted Quantile Sketch
- Sparsity-aware Split Finding
- Column Block for Parallel Learning
- Cache-aware Access
- Blocks for Out-of-Core Computation

본 내용은 XGBoost 의 여러 기능을 소개하는 논문인 XGBoost: A Scalable Tree Boosting System 을 바탕으로 한다.

2. Tree boosting in a nutshell

2.1 Regularized Learning Objective

CART(Classification and regression tree)

XGboost 의 Base Learner 는 CART 알고리즘이다. m 개의 변수와 n 개의 행으로 이루어진 데이터 $\mathcal{D} = \{(\boldsymbol{x_i}, y_i)\}$ ($|\mathcal{D}| = n, \boldsymbol{x_i} \in \mathbb{R}^m, y_i \in \mathbb{R}$) 에 대해 K 개의 additive function 을 이용한 tree ensemble 모델을 통해 output 을 예측하고자 한다. 이를 수식으로 표현하면,

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^{K} f_k(x_i), \qquad f_k \in \mathcal{F} \quad where\ \mathcal{F} = \{f(x) = w_{q(x)}\}(q: \mathbb{R}^m \to T, w \in \mathbb{R}^T)$$

이다. $\mathcal{F}$ 는 CART 로 알려진 회귀나무의 함수 공간이다. f_k는 트리 전체를 의미하고 각 트리에서 나온 결과값을 더하면 $\hat{y}_i$가 된다. 여기서 q 는 structure of each tree 이며 T는 number of leaves 를 의미한다. w는 leaf weight 이자 i 번째 leaf 의 score 즉 예측 값이다. 각각의 f_k는 w 와 q로 이뤄져 있다. 회귀나무에서 최종 예측치는 leaves 의 score 를 더하여 계산된다.

이를 그림으로 표현하면 아래와 같다.

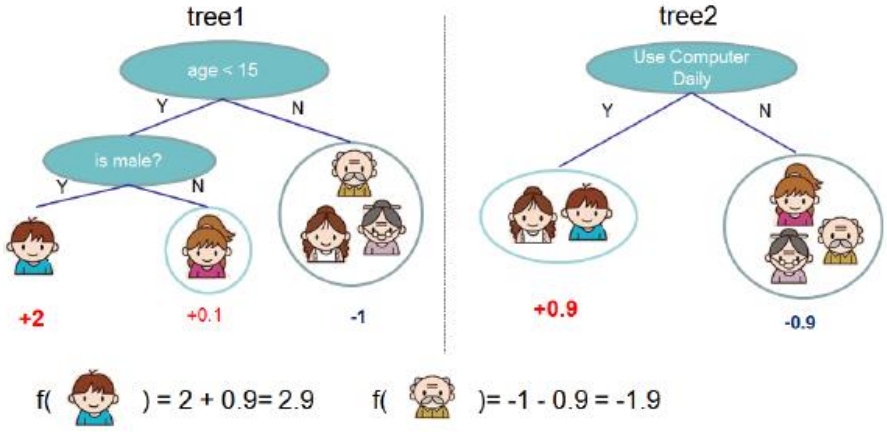


그림 1 Tree Ensemble Model 구조 예시

Loss function (How to evaluate q?)

그렇다면 각각의 회귀나무는 어떻게 구해지는가? 즉 $\sum_{k=1}^{K} f_k(x_i) = f_1(x_i) + f_2(x_i) + \cdots + f_K(x_i)$에서 $f_k(x_i)$(새로운 나무)가 뽑히는 기준은 무엇일까? XGboost 에서 impurity 를 낮추는 트리를 생성하기 위해 사용하는 regularized objective function 은 다음과 같다.

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \qquad where \quad \Omega(f_k) = \gamma T + \frac{1}{2}\lambda ||w||^2$$

해당 식에서 $\sum_i l(\hat{y}_i, y_i)$는 $\hat{y}_i$와 y_i의 차이를 계산하는 loss function 이고 $\sum_k \Omega(f_k)$은 regularization 을 담당하는 penalty term 이다. Penalty term $\Omega(f_k)$에서 T 는 terminal node 의 개수이고, γ는 트리의 terminal node 개수가 많아지는 경우, 즉 트리가 깊어지는 경우에 대해 penalty 를 부과하는 정도를 결정하는 parameter 이다. γT term 을 통해 모델의 복잡도에 대한 penalty 가 주어진다고 할 수 있다. 그 다음으로 $\frac{1}{2}\lambda||w||^2$에서 w 는 트리에서 각 leaf 의 예측값을 뜻하며, leaf weight 라고도 한다. 해당 term 이 과도하게 커질 경우 한 개의 트리가 전체 예측값에 큰 영향을 끼칠 수 있으므로, 큰 값이 계산되었을 때 tuning parameter λ를 통해 이에 대한 penalty 를 부과한다.

gradient tree boosting (How to choose w_j^*?)

하지만 위의 regularized objective function(loss function)은 ϕ(함수)에 대한 함수이기 때문에 표준적인 유한 차원 즉 Euclidean space 내에서 optimization 이 불가능하다. 이는 아래의 과정을 거쳐서 해결할 수 있다.

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

$$\mathcal{L}(\phi) = \sum_i l\left(\hat{y}_i^{t-1} + f_t(x_i), y_i\right) + \sum_k \Omega(f_k) \qquad \because \quad \hat{y}_i = \hat{y}_i^{t-1} + f_t(x_i)$$

이때 Taylor expansion(미지의 함수 $f(x)$를 다항함수로 근사시키는 것)을 적용하면,

$$l\left(\hat{y}_i^{t-1} + f_t(x_i), y_i\right) = F(X + \Delta X) \approx F(X) + F'(X)\Delta X + \frac{1}{2}F''(X)\Delta X^2$$

$= l\left(\hat{y}_i^{t-1}, y_i\right) + \partial_{\hat{y}_i^{t-1}} l\left(\hat{y}_i^{t-1}, y_i\right) f_t(x_i) + \frac{1}{2}\partial^2_{\hat{y}_i^{t-1}} l\left(\hat{y}_i^{t-1}, y_i\right) f_t(x_i)^2$ 이며,

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n}[l(\hat{y}_i^{t-1}, y_i) + g_i f_t(x_i) + \frac{1}{2}h_i f_t(x_i)^2] + \sum_k \Omega(f_k) \quad where\ \Omega(f_k) = \gamma T + \frac{1}{2}\lambda||w||^2$$

이다. 마지막 줄의 첫 항은 constant term 이므로 제거하고 이를 다시 쓰면,

$$\widetilde{\mathcal{L}^{(t)}} = \sum_{i=1}^{n}\left[g_i f_t(x_i) + \frac{1}{2}h_i f_t(x_i)^2\right] + \gamma T + \frac{1}{2}\lambda\sum_{j=1}^{T} w_j^2$$

$$= \sum_{j=1}^{T}\left[\left(\sum_{i\in I_j} g_i\right)w_j + \frac{1}{2}\left(\sum_{i\in I_j} h_i + \lambda\right)w_j^2\right] + \gamma T$$

단, $I_j = \{i|q(x_i) = j\}$: Leaf j 에 있는 instances

q : 데이터들을 Leaf 로 전달하는 Tree 의 구조

즉, $\{i|q(x_i) = j\}$ 는 Tree q 에 x_i 를 넣으면 j $index$에 배정된다는 뜻

두번째 줄에서 전체를 n 이 아닌 T 로 함수를 묶어준 이유는, loss function 이 tree boosting 이 ϕ 에 관한 함수이기 때문이다. $\sum_{i=1}^{n} g_i f_t(x_i)$와 $\sum_{j=1}^{T}\left(\sum_{i\in I_j} g_i\right)w_j$이 같은 이유는 그림 2 와 같다.

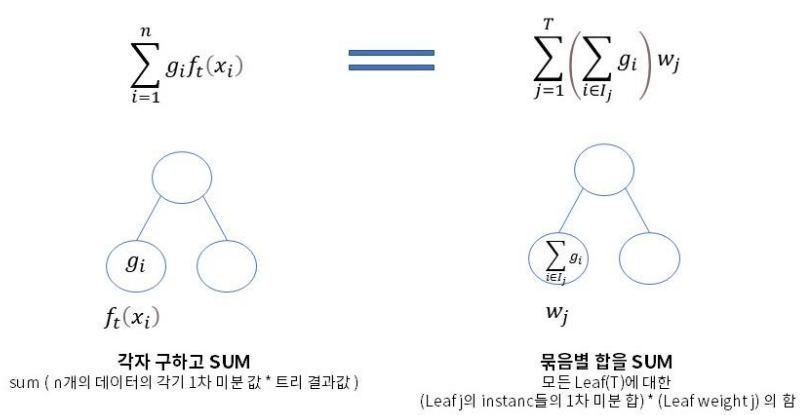


그림 2 두 식이 같은 이유에 대한 그림 설명

$q(x)$ 가 주어져 있다고 할 때, optimal weight w_j^*는,

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

이다. 또한 w_j^*를 대입한 최적 목적함수는 $\widetilde{\mathcal{L}^{(t)}} = -\frac{1}{2}\frac{\left(\sum_{i \in I_j} g_i\right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$ 이다.

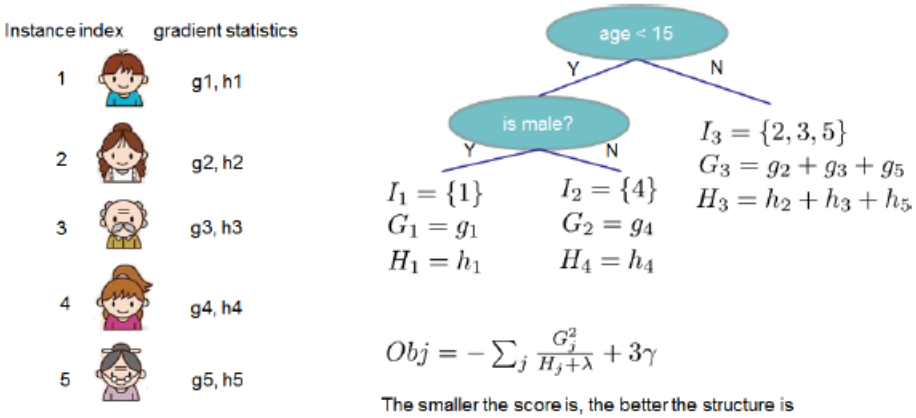


그림 3 최적 목적함수 계산 과정 예시

Regression example

이를 더 자세히 이해하기 위해 Regression 을 예시로 설명하고자 한다.

Regularized objective function 에 Taylor expansion 을 적용하여 식을 정리하면 다음과 같은 optimal weight w_j^*을 얻을 수 있었다. i 는 j 번째 leaf 에 속하는 데이터를 뜻한다.

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

Regression 문제에서 가장 흔하게 쓰이는 loss function 은 $l(\hat{y}_i, y_i) = (y_i - \hat{y}_i)^2$이다. 이를 이용하여 g_i, h_i를 구하면 다음과 같다.

$$g_i = \partial_{\hat{y}_i^{t-1}} l(\hat{y}_i^{t-1}, y_i) = \partial_{\hat{y}_i^{t-1}} (y_i - \hat{y}_i)^2 = -2(y_i - \hat{y}_i)$$

$$h_i = \partial^2_{\hat{y}_i^{t-1}} l(\hat{y}_i^{t-1}, y_i) = \partial^2_{\hat{y}_i^{t-1}} (y_i - \hat{y}_i)^2 = 2$$

이를 optimal w_j^*에 대입하면 다음과 같다.

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} + \gamma T = \frac{\sum_{i \in I_j} 2(y_i - \hat{y}_i)}{\sum_{i \in I_j} 2 + \lambda} + \gamma T$$

$$= \frac{\sum_{i \in I_j} (y_i - \hat{y}_i)}{\sum_{i \in I_j} 1 + \lambda} + \gamma T = \frac{Sum\ of\ Residuals}{Number\ of\ Residuals + \lambda} + \gamma T$$

따라서 optimal weight 는 residual 의 합과 residual 의 개수가 반영된 값이라고 할 수 있다.

greedy algorithm

일반적으로 가능한 모든 트리 구조 q를 나열하는 것을 불가능하다. 하나의 노드를 분할했을 때 왼쪽, 오른쪽으로 분류되는 집합을 각각 I_L, I_R이라고 하면, 노드 분할 후 손실의 감소량을 다음과 같다. Tree 를 Greedy 하게 분할할 때 아래의 식을 통해 split candidates 를 평가하게 된다.

$$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{\left(\sum_{i \in I_L} g_i\right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i\right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i\right)^2}{\sum_{i \in I_j} h_i + \lambda}\right] - \gamma$$

2.3 Shrinkage and Column Subsampling

Shrinkage and Column Subsampling 는 Regularized function 이외에도 over fitting 을 예방하는 두가지 방법이다. Shrinkage 는 tree 의 각 단계 이후 마다 η(learning rate)를 곱하여 각 tree 의 영향력을 낮춘다. 즉, 새롭게 추가된 가중치를 줄이는(scale) 것이다. Stochastic 최적화의 Learning Rate 와 유사하게, 이는 각 개별 tree 의 영향도를 감소시키고 모델을 향상시켜주는 미래의 tree 의 공간을 남긴다. Column Feature Subsampling 은 Random forest 처럼 split 을 나누는 feature 를 고를 때 모든 feature 를 다 고려하는 것이 아니라 일부 변수를 random 하게 고려하는 것이다.

3. Split finding algorithms

3.1 Basic exact greedy algorithm

tree learning 의 핵심 사항은 the best split 를 찾는 것이다. 분할탐색 알고리즘은 모든 feature 에서 가능한 모든 분할을 열거한다. Basic exact greedy algorithm 구조는 아래와 같다. 이때 연속적인 feature 의 모든 가능한 분할을 열거하는 것은 최적해를 보장해주지만 계산적인 부담이 크다. 이를 효율적으로 하기 위해서 알고리즘은 feature value 에 따라 데이터를 순서대로 정렬하고 정렬된 순서에 따라 gradient statistics 를 축적해야 한다.

Algorithm 1: Exact Greedy Algorithm for Split Finding

Input: I, instance set of current node
Input: d, feature dimension
$gain \leftarrow 0$
$G \leftarrow \sum_{i \in I} g_i$, $H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ ***to*** m **do**
  $G_L \leftarrow 0,\ H_L \leftarrow 0$
  for j *in sorted(I, by* $\mathbf{x}_{jk}$*)* **do**
    $G_L \leftarrow G_L + g_j,\ H_L \leftarrow H_L + h_j$
    $G_R \leftarrow G - G_L,\ H_R \leftarrow H - H_L$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
end
Output: Split with max score

알고리즘 1 Exact Greedy Algorithm for split finding

3.2 Approximate Algorithm

Exact greedy algorithm 은 모든 가능한 경우의 수를 고려하기 때문에 강력한 방법이지만, 데이터의 크기가 메모리가 감당하지 못할 정도일 경우 분할이 어려워질 수 있다. 이러한 단점을 극복하기 위해 XGboost 에서는 approximate algorithm 을 사용한다. 해당 알고리즘은 먼저 feature 분포의 percentiles 에 따라 후보 분할 지점을 제시하는 것이다. 해당 지점에서 통계량을 집계한 후 집계된 통계량을 바탕으로 만들어진 proposal 중 가장 좋은 해를 찾는다.

한편 Split 후보군을 언제 제시하는가에 따라 global variant 과 local variant 로 나뉜다. global variant 는 초기 트리 구성 단계에서 모든 후보 분할들을 제시하고 모든 수준에서 분할 찾기를 수행할 때 동일한 proposal 을 사용한다. local variant 는 매번 분할이 진행된 후에 다시 제시하는 방식이다. proposal 단계수로 보면 global 이 local 보다 수가 당연히 적다. 하지만 후보 분할지점 개수는 local 이 더 적다. 이는 global 에서는 매번 분할이 진행된 후에 후보 분할 지점이 개선되지 않기 때문이다. local 은 후보를 개선을 통해 deeper tree 에 적합할 가능성이 커진다.

Algorithm 2: Approximate Algorithm for Split Finding

for $k = 1$ ***to*** m **do**
  Propose $S_k = \{s_{k1}, s_{k2}, \cdots s_{kl}\}$ by percentiles on feature k.
  Proposal can be done per tree (global), or per split(local).
end
for $k = 1$ ***to*** m **do**
  $G_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} g_j$
  $H_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} h_j$
end
Follow same step as in previous section to find max score only among proposed splits.

알고리즘 2 Approximate Algorithm for split finding

3.3 Weighted Quantile Sketch

Approximate algorithm 의 중요한 단계 중 하나는 후보 분할 지점을 제시하는 것이다. 보통 feature percentiles 는 후보들이 데이터에 균등하게 분산되도록 하기 위해 사용된다. 하지만 여기서는 균등하게 나누지 않고 가중치를 준다. 먼저 각 train instance 에 대하여 k 번째 feature 값과 second order gradient statistics 의 muti-set 은 $D_k = \{(x_{1k}, h_1), (x_{2k}, h_2), \dots, (x_{nk}, h_n)\}$이다. 각 rank function 을 정의하면, $r_k: \mathbb{R} \rightarrow [0, +\infty]$

$$r_k(z) = \frac{1}{\sum_{(x,h)\in D_k} h} \sum_{(x,h)\in D_{k,}, x<z} h$$

이다. rank 는 feature k 가 z 보다 작은 instances 을 의미한다. 이때 목표는 분할지점 $\{s_{k1}, s_{k2}, \dots, s_{kl}\}$은 $|r_k(s_{k,j}) - r_k(s_{k,j+1})| < \epsilon, s_{k1} = \min_i x_{ik},\ s_{kl} = \max_i x_{ik}$ 을 통해 얻어진다. 이때 ϵ은 approximation factor 로서 percentile 을 얼마나 잘게 나눌지에 대한 파라미터이다. 직관적으로 이는 Candidate points 의 숫자와 연관되며, roughly 하게 설명하면 $\frac{1}{\epsilon}$ 개의 candidate points 가 존재함을 의미한다.

Weighted quantile sketch 는 각 quantile 마다 속한 data 개수가 같게 만드는 normal quantile 과 달리, quantile 에 속한 weight 의 합이 모두 같도록 데이터를 나누는 방법이다.

XGBoost 의 regularized loss function 식을 다음과 같이 정리할 수 있다.

$$\widetilde{\mathcal{L}^{(t)}} = \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(\mathrm{f_t})$$

$$= \sum_{i=1}^{n} \frac{1}{2} h_i \left(f_t(x_i) - \frac{g_i}{h_i} \right)^2 + \Omega(\mathrm{f_t}) + constant$$

이 식은 y 값이 $\frac{g_i}{h_i}$이고 weight 가 h_i일 때 weighted squared loss 값을 구하는 식과 같으므로, h_i를 weight 라고 할 수 있다. Weight 를 반영하는 rank function 은 다음과 같다.

$$r_k(z) = \frac{1}{\sum_{(x,h)\in D_k} h} \sum_{(x,h)\in D_{k,}, x<z} h$$

이 때 회귀 문제의 경우 loss function 의 2 차 미분값 h 는 모두 상수이다. 따라서 회귀 문제의 weighted quantile 은 normal quantile 과 같은 값을 가진다.

반면 분류문제에서 loss function 이 negative likelihood 일 경우, 즉 $L(y_i, F(x)) = -[y_i \log(p_i) + (1 - y_i) \log(p_i)]$일 때의 2 차 미분값은 $p_i(1 - p_i)$이므로 분류 문제의 weighted quantile 은 normal qunatile 과는 다른 값을 갖는다.

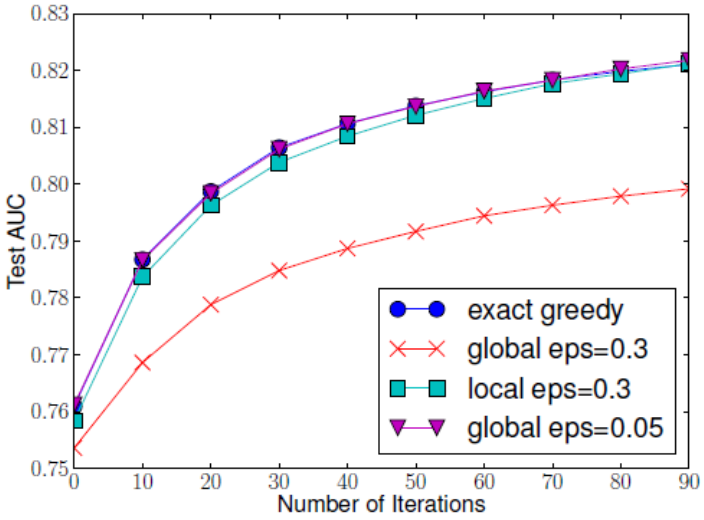


그림 4 global 과 local variant 에서의 epsilon 비교

그림 4 을 보면 local eps(ϵ)=0.3 과 global eps(ϵ) =0.05 일 때 exact greedy 선과 유사하다. local 은 eps(ϵ)가 global 보다 커도 exact greedy 선에 더 가깝다. 이는 local proposal 이 후보 분할 지점 개수를 각 split 마다 다시 제안하기 때문이다.

3.4 Sparsity aware Split Finding

실제로 많은 데이터에서 설명변수가 sparse 한 것은 흔한 일이다. sparsity 를 유발하는 여러가지 가능성이 있다. 데이터 내 결측 치가 존재하는 경우, 통계량에서 빈번한 0 entry 나타나는 경우, one-hot encoding 같은 feature engineering artifacts 가 진행되는 경우이다. 데이터 내 sparsity 패턴을 인식하는 알고리즘은 트리 노드 missing value 에 대한 default direction 을 제시하여 데이터의 sparsity 패턴을 알고리즘이 인지하도록 한다. 이를 수식으로 표현하면 아래와 같다.

Algorithm 3: Sparsity-aware Split Finding

Input: I, instance set of current node
Input: $I_k = \{i \in I | x_{ik} \neq \text{missing}\}$
Input: d, feature dimension
Also applies to the approximate setting, only collect statistics of non-missing entries into buckets
$gain \leftarrow 0$
$G \leftarrow \sum_{i \in I}, g_i, H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ ***to*** m **do**
  // enumerate missing value goto right
  $G_L \leftarrow 0,\ H_L \leftarrow 0$
  for j *in sorted*$(I_k$, *ascent order by* $\mathbf{x}_{jk})$ **do**
    $G_L \leftarrow G_L + g_j,\ H_L \leftarrow H_L + h_j$
    $G_R \leftarrow G - G_L,\ H_R \leftarrow H - H_L$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
  // enumerate missing value goto left
  $G_R \leftarrow 0,\ H_R \leftarrow 0$
  for j *in sorted*$(I_k$, *descent order by* $\mathbf{x}_{jk})$ **do**
    $G_R \leftarrow G_R + g_j,\ H_R \leftarrow H_R + h_j$
    $G_L \leftarrow G - G_R,\ H_L \leftarrow H - H_R$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
end
Output: Split and default directions with max gain

알고리즘 3 missing data 에 대한 sparsity aware split finding

이를 그림으로 설명하면, Sparse matrix 에 missing value 가 존재할 경우 해당 record 는 default direction 으로 자동적으로 분류된다. Sample 값들을 sorting 하여 두개의 노드로 분할할 때 한번은 missing value 를 전부 왼쪽으로 배치, 한번은 전부 오른쪽으로 배치하여 더 좋은 split point 를 찾는 쪽으로 설정한다.

그림 5 sparsity aware split algorithm 진행 과정

아래 그래프를 통해 실제로도 sparsity aware algorithm 을 사용했을 때 learning 시간이 다른 알고리즘에 비해 줄어들었음을 알 수 있다.

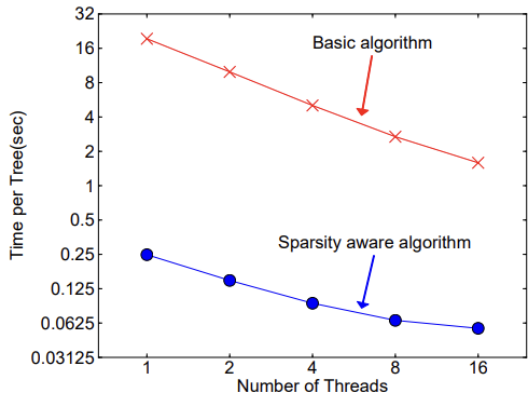


그림 6 Sparsity aware algorithm 와 Basic algorithm 의 time 비교

4. System Design

4.1 Column Block for Parallel Learning

데이터를 정렬하는 과정은 트리 모델 학습 과정에서 많은 시간을 차지한다. 계산 시간을 줄이기 위해 XGBoost 는 sorting 된 각 column 들을 block 이라는 단위로 저장한다. 이러한 block 단위 저장방식에 의해 변수 별 계산이 병렬적으로 가능해지고 이에 따라 계산시간이 줄어든다.

이를 보다 잘 이해하기 위해 기본 개념을 설명하고자 한다.

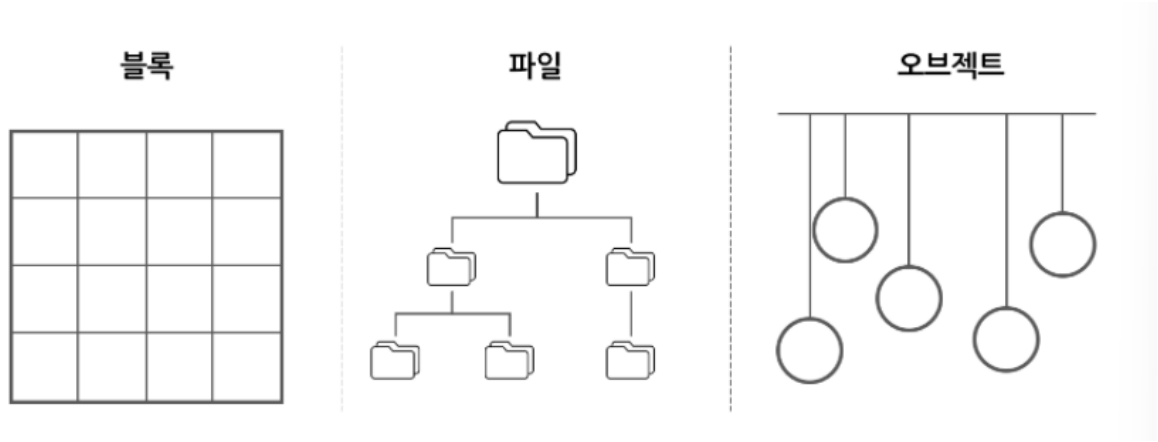
블록 단위 저장방식은 데이터를 보유하고 구성하는 방식인 스토리지 형식의 하나이다. 스토리지 형식에는 크게 블록, 파일, 오브젝트 스토리지가 있다.

블록 스토리지는 데이터를 일정한 크기의 단위로 분리하여 저장하는 방식이다. 각 블록은 고유한 위치 주소를 가져 데이터를 분산 저장했다가 하나로 재구성하기에 편리하다.

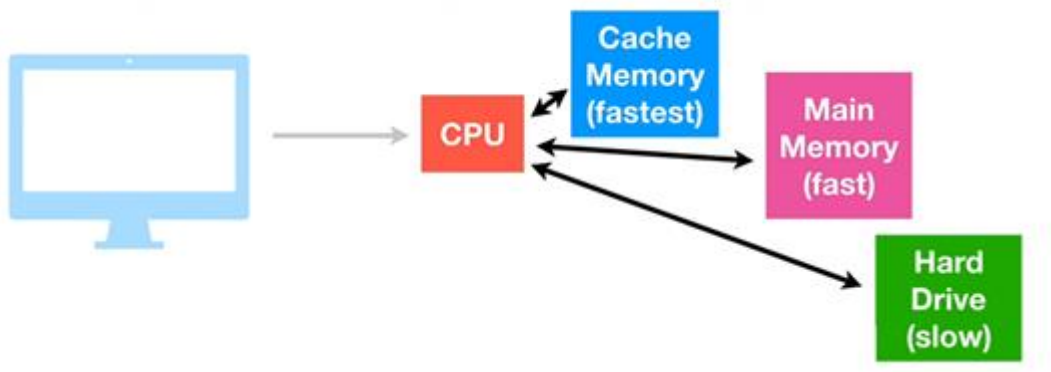
파일 스토리지는 데이터를 폴더 안에 저장하는 방식인데, 이 때 파일은 계층형 구조를 가진다.

오브젝트 스토리지는 데이터를 하나의 오브젝트라는 단위로 관리하는 방식으로, 폴더 속에 폴더가 들어갈수 있는 연속적인 형태의 파일 스토리지와 달리 평면적인 저장방식이다.

세 저장방식을 그림으로 표현하면 다음과 같다.

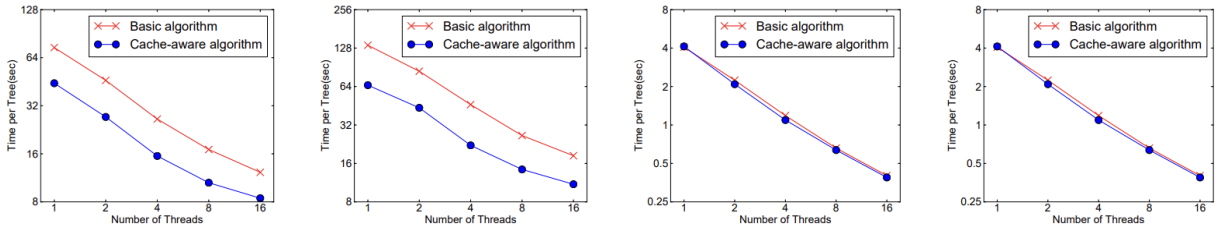


4.2 Cache-aware Access



컴퓨터의 CPU 에는 3 개의 메모리 공간이 존재하고, 각 메모리 공간은 계산 속도에 차이가 있다. XGBoost 는 3 개 중 속도가 가장 빠른 cache memory 의 활용을 최대화하여 계산 속도를 향상한다.

The exact greedy algorithm 에서, cache aware algorithm 은 basic algorithm 보다 계산 속도가 빠르다. large dataset 에 적용한 예시는 아래와 같다.



(a) Allstate 10M (b) Higgs 10M (c) Allstate 1M (d) Higgs 1M

그림 7 exact greedy algorithm 의 cache aware algorithm 계산 속도

Approximate algorithm 과정에서도 cache aware algorithm 을 통해 계산속도를 개선할 수 있지만, block size 가 너무 크면 cache access 에 문제가 발생할 수 있다. 반대로 block size 가 너무 작으면 병렬 학습 과정이 비효율적이다. 따라서 적절한 block size 를 선택하는 것도 중요하다. Approximate algorithms 과정에서 block size 를 다르게 설정한 그림 7 을 보면, 두 예시 모두 block size 가 너무 크거나 작지 않을 때 계산 시간이 짧다.

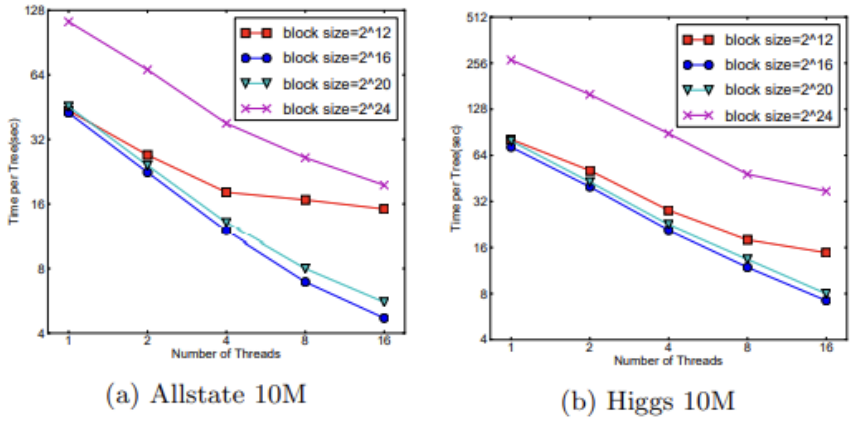


그림 8 approximate algorithm 의 cache aware algorithm 계산 속도

4.3 Blocks for Out-of-core Computation

크기가 큰 데이터를 disk 로부터 불러오는 시간을 줄이기 위해 데이터를 압축하는 block compression 방법과 데이터셋을 여러 disk 에 분산시키고 병렬로 처리하는 block sharding 방법을 이용한다.

5. Example using python and R

XGboost : parameter description

함수 설명

파이썬에서는 분류 문제는 XGBoostClassifier, 회귀 문제는 XGBoostRegressor 함수를 이용하여 XGBoost 모델을 실행할 수 있다. R 에서는 xgboost package 내의 함수를 이용한다.

함수를 사용할 때 조정할 수 있는 파라미터들은 다음과 같다. R 에서 함수 이름이 다른 경우 alias 에 표시하였다.

Parameter	default	설명
booster	gbtree	부스팅에 사용할 모델의 종류. (gbtree / gblinear / dart – tree boosting 에 dropout 을 적용)
colsample_bylevel	1	트리 내 각각의 depth(level)마다 사용할 column 의 subsample 비율
colsample_bytree	1	각각의 트리를 구성할 때 column 의 subsample 비율
colsample_bynode	1	각각의 node 에 대한 column 의 subsample 비율
gamma	0	loss function 에서 모델의 복잡도를 반영하는 정도를 결정하는 parameter
learning_rate (alias = eta)	0.3	이전 트리 정보에 대한 학습률
max_depth	6	트리의 최대 깊이
min_child_weight	1	child 에서 필요한 모든 관측치에 대한 가중치의 최소 합.
n_estimators(alias = nrounds)	100	부스팅을 위한 반복 학습 수
n_jobs	12	XGBoost 를 실행하기 위한 병렬 처리 개수
reg_alpha(alias = alpha)	0	L1 정규화 파라미터
reg_lambda(alias = lambda)	1	L2 정규화 파라미터
subsample	1	training instances 에서 subsample 의 비율
eval_metric	according to objective	모델 평가지표이다. rmse: root mean square error error - Binary classification error rate AUC - Area under curve (used in classification)
objective	reg:squarederror	reg:squarederror: regression with squared loss. logistic regression for binary classification.
tree_method[default=auto]		auto: Use heuristic to choose the fastest method. exact: Exact greedy algorithm approx.:Approximate greedy algorithm

depth 정의

위 표에서 depth 란, 분할을 처음 시작할 때 전체 데이터를 하나의 집합으로 보는 맨 윗부분을 제외하고 만들어지는 트리의 깊이를 말하며, 트리 구조 상 수평에 있는 노드들이 같은 레벨의 깊이에 속한다. 밑의 예시 그림에서 트리의 depth 는 2 다.

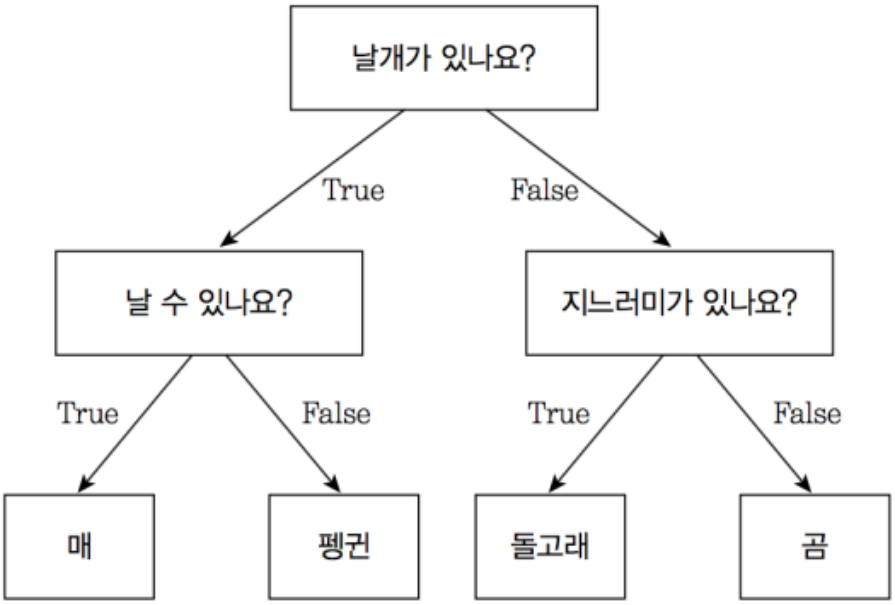


그림 9 depth 설명 그림 예시

colsample_bylevel/ colsample_bytree/ colsample_bynode 의 차이

사용할 columns 의 sample 비율을 결정하는 파라미터이다. 이 때 colsample_bytree 의 경우 하나의 트리가 만들어질 때 그 비율에 맞춰 사용할 columns 가 random 하게 선택되고, 선택된 columns 들이 해당 트리를 만들 때 사용된다. 반면 colsample_bylevel 은 한 트리 안에서 매 depth 마다 새로 sampling 할 columns 의 비율을 설정하고, depth 마다 새로 정해진 비율에 맞춰 columns 를 sampling 할 때 설정하는 파라미터이다. colsample_bynode 는 각 노드에 대한 columns 의 subsample 비율이다.